AUTOMATALAB

User Manual

S-CSPC327

AUTOMATA THEORY AND FORMAL LANGUAGES

Finals Project

Developed by:

Dancel, Ryan Justin

Delos Reyes, Dwyn Anthony

Simera, Kurt Miguel

Trance, Marti Kier

Submitted to:

Ma'am Jennylinde Manaois

BCS32

Table of Contents

I. Introduction to AutomataLab **3**

II. Getting Started **3**

III. The Expression & Model Selector **3**

Understanding the Three-Column Layout 4

1st Column: Selecting a Regular Expression 4

2nd Column: Choosing a Computational Model 5

3rd Column: Model Description & Formal Definition 5

IV. Regex & Automata Configuration **6**

- A. String Input Fields 6
- B. Status Indicators 7
- C. Action Trigger 8

V. Visualizing the Machine **8**

VI. Troubleshooting **9**

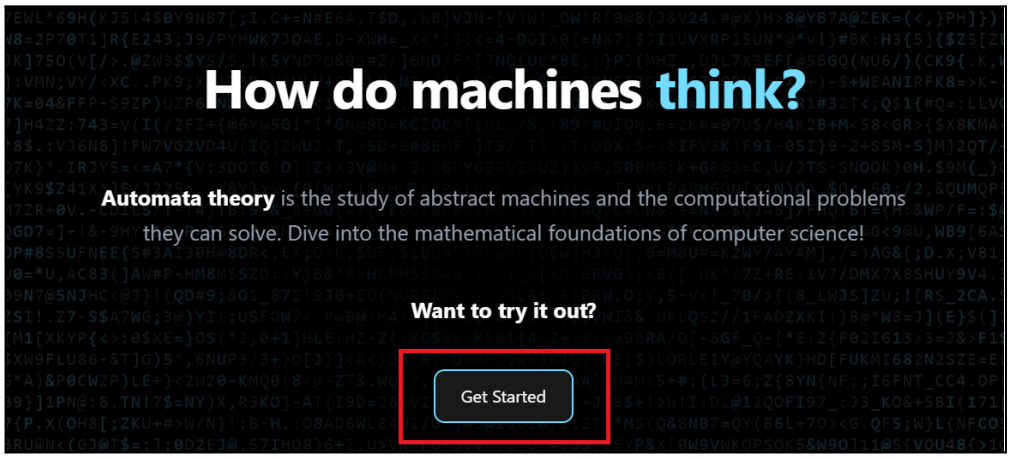
I. Introduction to AutomataLab

AutomataLab is an educational tool designed to help you visualize how abstract machines process data. By converting complex Regular Expressions (Regex) into visual models, you can explore how logic translates into state-based transitions.

II. Getting Started

Upon landing on the homepage, you will be greeted by the "How do machines think?" introduction.

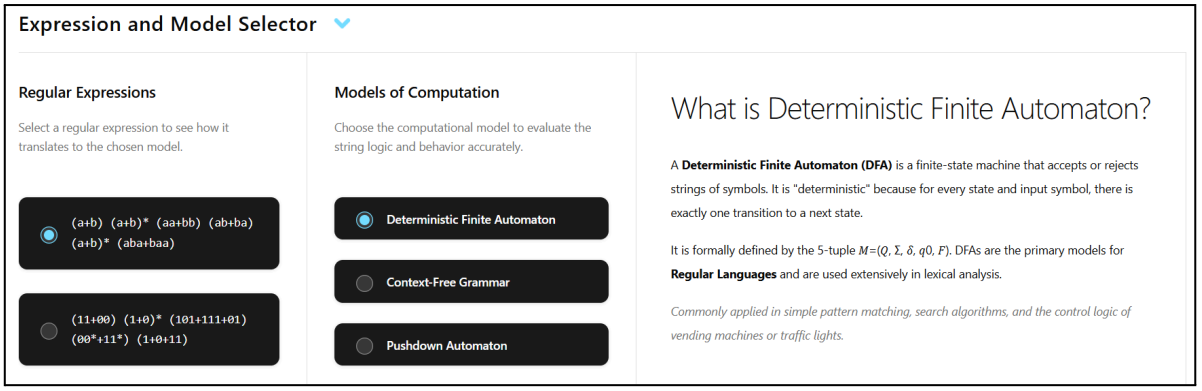
- Click the **[Get Started]** button to jump directly to the Automata Theory Playground.



III. The Expression & Model Selector

The Expression and Model Selector serves as the brain of the lab. It is located directly below the header and is organized into three distinct columns to streamline your configuration process.

> **Navigation Tip:** To the right of the section title, you will notice a downward-pointing arrow icon. This is a Quick-Jump anchor designed to snap the interface directly to the configuration area, saving you from manual scrolling and keeping your focus on the logic.



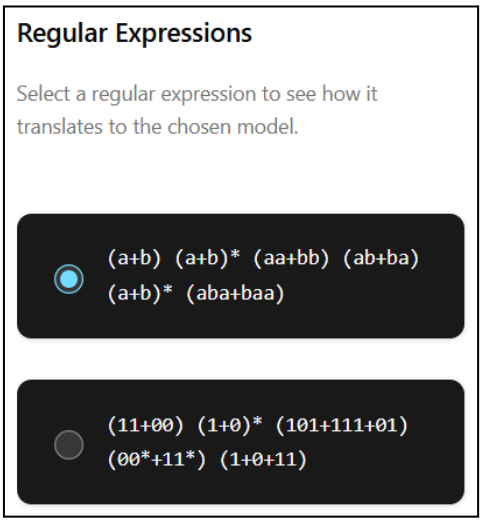
Understanding the Three-Column Layout

The interface is split into three functional areas to help you move from abstract logic to specific machine behavior.

1st Column: Selecting a Regular Expression

In the **Regular Expressions** panel, choose from a variety of pre-set expressions. These range from simple binary logic to complex string patterns.

- **Function:** By clicking on a regex (such as `(a+b)*` or `(11+00)`), you are setting the rules for the language you want the machine to recognize.
- **Selection:** The active expression will be highlighted, indicating it is the current logic being fed into the system.

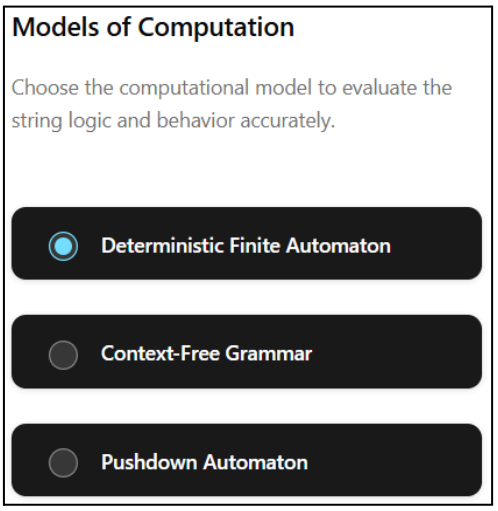


2nd Column: Choosing a Computational Model

Once your expression is set, select a **Model of Computation** to see how that logic is handled by different theoretical machines:

- **DFA (Deterministic Finite Automaton):** Best for standard pattern matching.
- **CFG (Context-Free Grammar):** Best for understanding the rules and structure of languages.
- **PDA (Pushdown Automaton):** Adds memory (a stack) to the machine.

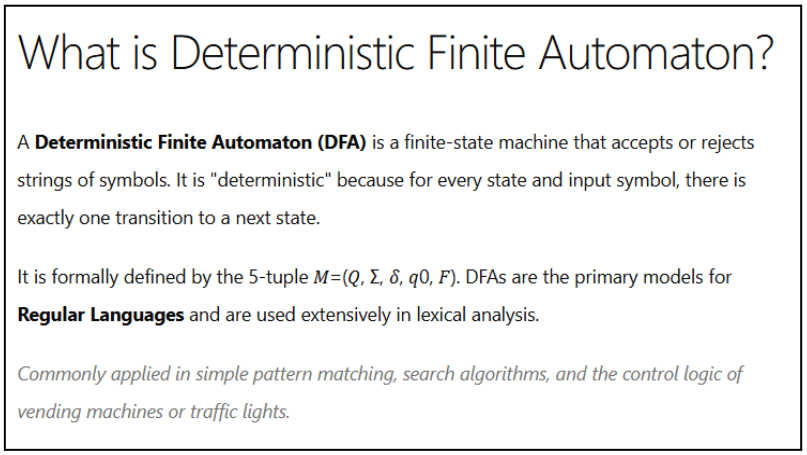
You can select between a **DFA**, a **CFG**, or a **PDA**. Changing this selection will instantly prepare the third column and the visualizer to reflect that specific machine's architecture.



3rd Column: Model Description & Formal Definition

The final column provides academic context for your selection in Column 2.

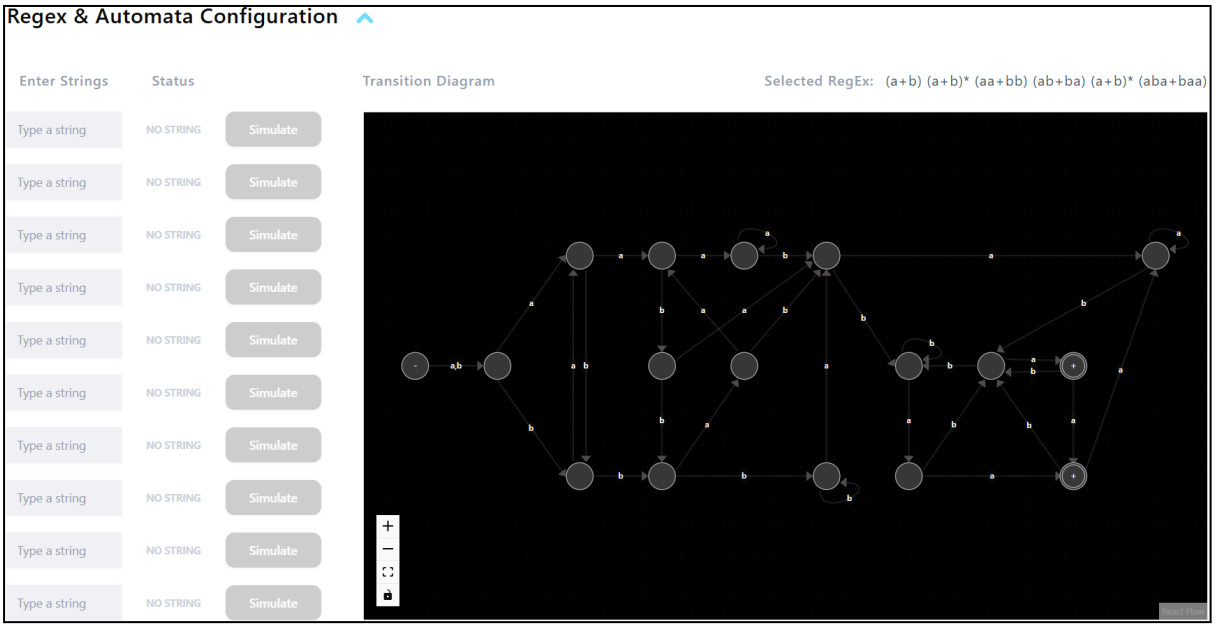
- **Function:** It dynamically updates to show a concise summary of the selected model.



IV. Regex & Automata Configuration

The **Regex & Automata Configuration** section allows you to interact with your chosen model in real-time.

> **Navigation Tip:** To the right of the section title, you will notice an upward-pointing arrow icon. This is a Quick-Jump anchor designed to snap the interface directly to the selector area, saving you from manual scrolling and keeping your focus on the logic.

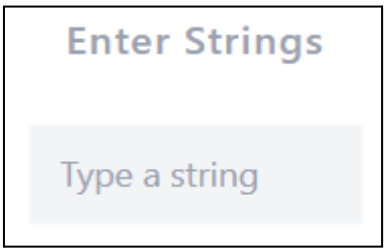


The simulation area is where the mathematical theory meets user interaction. Here is how to navigate the validation process:

- **A. String Input Fields**

  Located under the Enter Strings column. This is where you provide the raw data (the strings) that you want the machine to evaluate.

  - User Action: Type your binary or alphabetic characters here.



- **B. Status Indicators**

  Located under the Status column, directly to the right of each input field. These are the real-time feedback labels that tell you if your string is valid according to the machine's logic.

  - **NO STRING (Gray/Neutral):** Indicates the field is empty, and the machine is idle.

    NO STRING

  - **READY (Brown):** The input field contains a string, but the simulation has not been executed yet.

    User Action: This is your cue to click the **Simulate** button. The machine has read your input and is standing by to map the transition path.

    READY

  - **PROCESSING (Pulsing Blue):** The machine is currently calculating the path through the states or grammar rules.

    Visual Cue: The pulsing effect indicates active computation. For Regex, CFG, PDA, this shows the system is working to determine if the string reaches a Final State.

    PROCESSING

  - **ACCEPTED (Green):** Indicates the string successfully reached a Final State in the DFA or PDA.

    VALID

  - **REJECTED (Red):** Indicates the string led to a non-accepting state or broke the grammar rules.

    INVALID

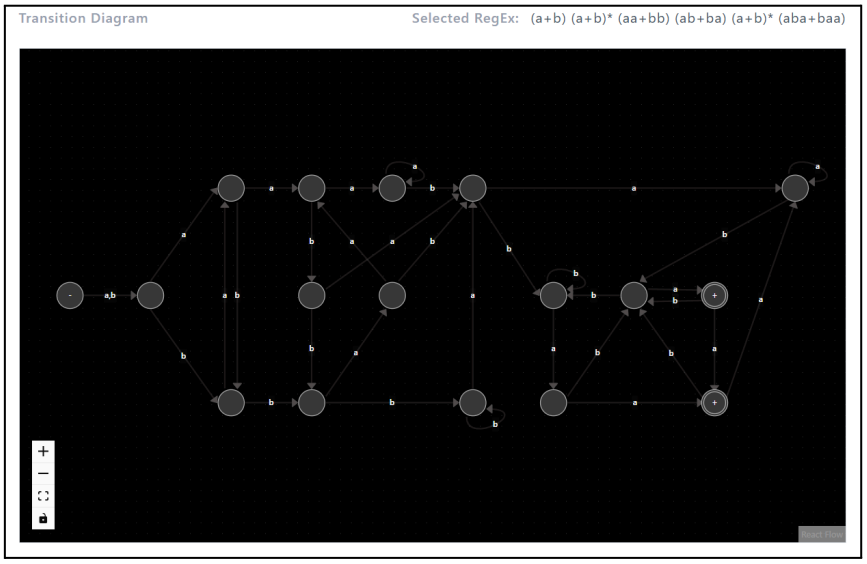
- **C. Action Trigger**

  The **Simulate** button. Clicking this confirms the input and triggers the visual "path" to be highlighted in the Transition Diagram.

V. Visualizing the Machine

The most powerful feature of **AutomataLab** is the **Transition Diagram**. This interactive window provides a live visual map of the logic you have configured.

- The Transition Diagram, powered by React Flow, serves as a live, synchronized map of the machine's logic. It is fully adaptive, meaning the entire layout and structural format intelligently rebuild themselves in real-time to match the specific rules of your chosen model, whether it is a DFA, CFG, or PDA. Any change to your Regular Expression or Model type triggers an instant refresh, ensuring the visual output remains perfectly in sync with your current configuration.



VI. Troubleshooting

While AutomataLab is designed to be reactive, you may occasionally encounter a synchronization lag if complex logic is switched too rapidly.

- If the Transition Diagram does not appear to be updating with your selected model, or if the Status Indicators seem stuck in the Processing state, simply **refresh the page (F5 or the browser refresh button)**.

Conclusion

AutomataLab was developed to bridge the gap between abstract automata theory and practical visualization. We hope this tool provides a clearer understanding of how Deterministic Finite Automata, Context-Free Grammars, and Pushdown Automata process languages and logic.

Development Team

For technical inquiries or feedback regarding the AutomataLab platform, please contact the Group 3 (BCS32) development team:

- Dancel, Ryan Justin
- Delos Reyes, Dwyn Anthony
- Simera, Kurt Miguel
- Trance, Marti Kier

S-CSPC327: Automata Theory and Formal Languages